Container-State-Aware Energy Consumption Minimization for Serverless Functions

Jinquan Zhang, Han Cao, Long Chen, Siyuan Li, Xiaoping Li, *Senior Member, IEEE*, and Bing Ai

***Abstract*—In this paper, we investigate the problem of minimizing the total energy consumption for serverless functions (SFs) while ensuring that the cold start probability of each SF remains below a specified threshold. In serverless computing, containers are reused to reduce the occurrence of cold starts, which increases the static energy consumption and decreases the dynamic energy consumption of physical machines (PMs). The trade-off between cold start reduction and energy consumption introduces significant challenges, especially considering that container states (idle, cold start, and running) have distinct energy characteristics. To address these challenges, we propose a container-state-aware energy consumption model that accurately captures the dynamic energy characteristics of PMs. Furthermore, we develop an enhanced Bayesian Optimization-Based Energy Consumption Minimization (BOECM) algorithm, which determines the optimal reuse times for different serverless functions (SFs) by decomposing the high-dimensional search space into a series of lower-dimensional sub-problems. In addition, several tailored heuristic strategies are proposed for request allocation and container deployment. The proposed BOECM algorithm is compared with several existing heuristic and meta-heuristic algorithms for similar problems. Experimental results demonstrate that the proposed method significantly reduces total energy consumption, maintains cold start probabilities within acceptable limits, and outperforms the baselines.**

***Index Terms*—Cold start, serverless, energy consumption, reuse time, Bayesian Optimization.**

I. Introduction

SERVERLESS computing has been widely adopted in many real-world scenarios, such as real-time data processing, AI reasoning, video transcoding,[1] etc. The primary motivation is that it enables developers to concentrate on business logic, i.e., serverless functions (SFs), without the need to manage servers. From the perspective of cloud providers, requests for these SFs are commonly executed on containers. When an SF is invoked and no idle container of this SF is available, a new container is launched. The above process is a process known as a cold start, which includes creating containers, loading function codes, initializing the runtime environment, etc. In many cases, the time for cold start is comparable to the time of request execution [1], [2], that is, several seconds. Consequently, cold starts result in significant request execution latency. This becomes more serious when considering SFs with high real-time demands, such as video surveillance systems [3] and multimedia stream processing [4]. Therefore, reducing the occurrence of cold starts is crucial to ensuring the quality of services.

Cloud providers widely adopt reuse as one of the approaches to mitigate cold starts. This approach involves keeping a container alive for a specific duration after the last request it receives is finished, known as the reuse time, instead of releasing it immediately. The container can be reused without experiencing a cold start if a new request is received within this duration. Although reuse effectively reduces the probability of cold starts, it significantly affects total energy consumption. Containers of various SFs are hosted on physical machines (PMs), whose energy consumption can be divided into static (baseline power for staying active) and dynamic (power due to function execution and cold starts). The dynamic energy consumption of a cold start is approximately 70% of that of the function execution [5], while the dynamic energy consumption of the idle containers is negligible. As the reuse time increases, the static energy consumption increases, whereas the dynamic energy consumption decreases since the probability of cold starts decreases. Therefore, determining the appropriate reuse times for SFs is crucial to minimize total energy consumption.

In this paper, we study the problem of minimizing the total energy consumption while maintaining the cold start probabilities of SFs below a specified threshold. Historical data recording the arrival and execution times of function invocations is used to search for the optimal reuse times. Containers are deployed on homogeneous PMs. Containers created for the same SF share identical CPU and memory configurations, while those of different SFs may differ. The challenges of this problem are summarized as follows: i) Different states of containers significantly influence the total energy consumption of PMs and the serverless platform. It is a challenge to set up the energy consumption model considering the different states of

Received 25 May 2025; revised 25 January 2026; accepted 11 March 2026. Date of publication 16 March 2026; date of current version 12 June 2026. This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFB3305500, in part by the National Natural Science Foundation of China under Grant 62273089, Grant 62102080, and Grant 62306099, in part by the Key Scientific Research Foundation of the Education Department of Anhui under Grant 2023AH052178, and in part by the Fundamental Research Funds for the Central Universities under Grant 2242022R10017. *(Corresponding author: Xiaoping Li.)*

Jinquan Zhang and Xiaoping Li are with the School of Computer Science, Guangdong University of Technology, Guangzhou 510006, China (e-mail: zjq8860@gdut.edu.cn; xpli@gdut.edu.cn).

Han Cao, Long Chen, and Siyuan Li are with the School of Computer Science and Engineering, Southeast University, Nanjing 211189, China, and also with the Key Laboratory of New Generation Artificial Intelligence Technology, Interdisciplinary Applications, Southeast University, Ministry of Education, Nanjing 211189, China.

Bing Ai is with the School of Artificial Intelligence and Big Data, Hefei University, Hefei 230601, China.

Digital Object Identifier 10.1109/TSC.2026.3674023

[1]https://www.aliyun.com/product/fc

1939-1374 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:21:50 UTC from IEEE Xplore. Restrictions apply.

the containers. ii) The total energy consumption is affected by the reuse times of all SFs. It is difficult to find the optimal reuse time for each SF efficiently in a continuous high-dimensional decision space. iii) Requests are executed on containers, while containers are deployed on PMs with limited resource capacities. How to allocate requests and deploy containers is challenging.

Existing work on energy minimization for SFs primarily focuses on the resource consumption of individual functions (e.g., the occupied CPU and memory), which fails to accurately reflect the actual energy consumption PMs. Even when the energy consumption of PMs is considered, their dynamic energy consumption is assumed to be proportional to the CPU utilization, ignoring the variations caused by different container states. To address these limitations, we propose a container-state-aware energy consumption model and a Bayesian Optimization-Based Energy Consumption Minimization (BOECM) algorithm. The main contributions of this work are summarized as follows:

1) We develop a container-state-aware energy model that considers dynamic power consumption under different container states: idle, cold start, and running, resulting in a more realistic estimate.
2) To address the high-dimensional decision space, we decompose the reuse time optimization into subproblems, each with a single variable, and solve them using an improved Bayesian Optimization approach with minimal iterations.
3) We design a set of tailored heuristic strategies for request allocation and container deployment and identify the optimal combination of strategies for the problem under study.

The rest of this paper is organized as follows. Section II reviews the related work. The problem is described and formulated in Section III. Section IV introduces the proposed BOECM algorithm. The experimental results are shown in Section V, followed by the conclusions and future works in Section VI.

II. Related Work

This section reviews previous research related to our work, including cold start mitigation strategies, energy minimization scheduling, request allocation and container deployment in serverless computing platforms.

A. Cold Start Mitigation Strategies

A wide range of strategies have been proposed to mitigate cold start delays by avoiding cold starts or reducing cold start latency, and these strategies are often used in combination.

Reuse and pre-warming are two commonly adopted approaches to avoid cold starts. Major cloud providers such as AWS Lambda,[2] Google Cloud Functions,[3] and Microsoft Azure[4] implement reuse by retaining containers for several minutes after execution. Mahmoudi et al. [6] developed an analytical model based on reuse to calculate the probability of cold start and the average number of idle containers. However, the assumption of Poisson distributed arrivals may not reflect real-world workloads, which can be periodic [7] or bursty [8]. In addition, reuse-time optimization was not addressed. Jia et al. [5] proposed the RAEF algorithm, which sets reuse times based on the cold start power, the cold start duration, and the idle power.

Unlike reuse, **pre-warming** initializes containers ahead of anticipated demand peaks. Shahrad et al. [9] proposed the Hybrid Histogram Policy (HHP), using invocation time histograms to calculate reuse and pre-warming durations at the 99th and 5th percentiles, respectively, extended by 10% to handle prediction errors. Vahidinia et al. [10] introduced a two-layer adaptive framework based on reinforcement learning to optimize reuse time and the number of pre-warmed containers. Gias et al. [11] used a layered queueing network (LQN) model to plan capacity, optimize CPU allocation, and reuse time. In [12], CPU cores are shared among containers with soft limits to improve efficiency. Zhang et al. [13] combined horizontal and vertical scaling to reduce costs while meeting delay constraints. Hu et al. [14] mitigated cold starts by using an RNN-based predictive pre-warming framework that forecasts future function invocations and dynamically adjusts resource provisioning to balance responsiveness and resource efficiency. Cai et al. [15] introduced priority-aware scheduling, using workload prediction and reinforcement learning to preferentially prewarm and schedule high-impact functions to minimize overall response time. In edge computing scenarios, Golec et al. [16] proposed MASTER, a machine-learning-based framework that predicts cold start latency in serverless edge computing, enabling proactive and resource-efficient scheduling. Choupani et al. [17] reduced cold starts in serverless edge computing by jointly applying Q-learning–based resource autoscaling and heuristic warm-instance scheduling to adaptively balance QoS and resource efficiency under dynamic workloads.

Several other works aim to **reduce cold start latency**. Xu et al. [18] proposed language-specific pre-launched container pools to speed up container creation and runtime initialization, although code loading remains a bottleneck. Liu et al. [19] reduced deployment overhead by loading code on demand rather than eagerly loading all functions. Akkus et al. [20] proposed SAND, an application-level sandbox that loads all SFs of the same application in a shared container. Furthermore, dependencies among SFs can be leveraged to mitigate cold starts. Daw et al. [21], Shen et al. [22], and Bhasi et al. [23] proposed various strategies to preemptively prepare containers based on function workflows and predicted execution paths.

B. Energy-Aware Scheduling

Energy consumption plays a crucial role in data center operational costs [24]. Most of the existing work on cold starts focuses on balancing resource consumption (e.g. CPU, memory) and the probability of cold start [9], [10], [11], [18], [21]. However, relatively few studies explicitly model or optimize the total energy consumption of serverless platforms.

Yang et al. [25] attempted to minimize the latency of SF in a power budget. Gunasekaran et al. [26] and Bhasi et al. [23] evaluated energy consumption at the cluster level, but did not

[2]https://aws.amazon.com/cn/lambda
[3]https://cloud.google.com/functions
[4]https://azure.microsoft.com

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:21:50 UTC from IEEE Xplore. Restrictions apply.

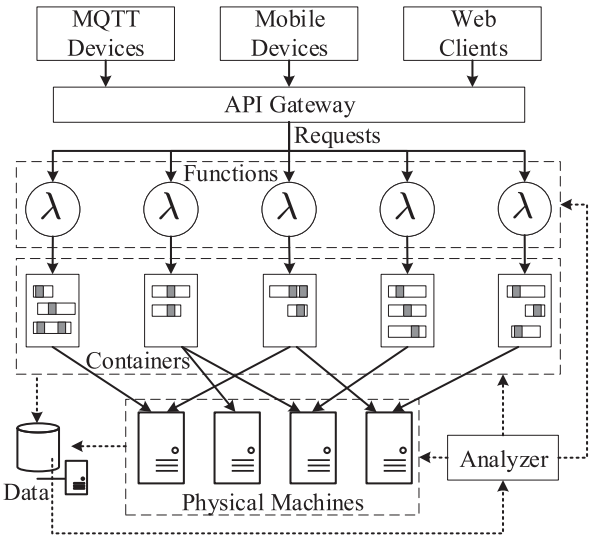


Fig. 1. System architecture.

explicitly model PM energy consumption. Jia et al. [5] highlighted the energy differences between the cold start, idle, and running container states, but focused only on minimizing energy consumption during the running phase through resource configuration. Rastegar et al. [27] minimized the energy cost of executing function chains using DVFS techniques, though their method applies only to single-core systems.

C. Request Allocation and Container Deployment

Compared to virtual machine-based cloud platforms, request allocation and container deployment in serverless systems have received relatively limited attention. Mahmoudi et al. [6] allocated requests to the most recently created container. In AWS Lambda, container deployment is treated as a bin-packing problem to optimize memory usage [2]. Gunasekaran et al. [26] proposed assigning requests to the container with the fewest idle slots and deploying containers to PMs with the least available CPU cores. In [28], requests were assigned to new containers if the cold start delay was shorter than the expected queueing time, with containers deployed using a first-fit strategy. Mampage et al. [29] adopted a round-robin request allocation strategy and applied reinforcement learning for mapping containers to virtual machines. Choupani et al. [17] proposed request allocation strategies based on warmest-container-first, random, round-robin, and coldest-container-first policies. These strategies provide useful insights for the request allocation mechanisms considered in this work.

D. Summary

In summary, the existing studies mainly focus on making a trade-off between CPU/memory consumption and cold start probability for a single function. Little attention is paid to the energy consumption of PMs in serverless computing. No studies consider the energy consumption power of PMs in different container states. To the best of our knowledge, no existing work incorporates a container-state-aware energy model that distinguishes between idle, cold start, and running states, nor integrates such a model into reuse time optimization, request allocation, and container deployment. In contrast, our work presents a container-state-aware energy minimization framework while ensuring the cold-start probability. To emphasize our focus on energy-aware reuse-time optimization, we adopt a simple reactive scaling mechanism. Although a variety of prediction-based autoscaling approaches have been proposed to reduce cold starts by introducing moderate additional resource consumption, our approach is complementary to these strategies and can be naturally integrated with them in future systems.

III. Problem Formulation

This section formulates the problem of minimizing energy consumption in a serverless computing platform, while satisfying cold start constraints and physical resource limitations.

A. System Overview

The architecture of the system is illustrated in Fig. 1. Incoming requests from various client devices are routed through an API Gateway to trigger the corresponding SFs. Requests are executed by containers, each associated with a specific SF, and containers of the same SF share a common reuse time. Each request is executed by a single container. When no idle container of the corresponding SF is available, a new container is created, and the request is executed only after the container initialization is completed. For simplicity, request queuing, delay, and rejection are not explicitly modeled. The deadline and QoS constraints are not explicitly modeled, as this work focuses on cold-start probability as the primary performance constraint. These containers are deployed on physical machines (PMs), and all relevant execution metadata, including request arrival and execution times, container creation timestamps, container states, and deployment locations, are recorded and analyzed. An offline analyzer leverages these historical data to determine the optimal reuse time for each SF (where the proposed BOECM algorithm is integrated), and to refine request allocation and container deployment strategies accordingly.

B. Assumptions

For simplification, the following assumptions are made to simplify the modeling.

- The cloud has sufficient idle PMs, and the startup time of PMs is ignored.
- There is no limit on the number of containers that can be created per function.
- The CPU and memory configurations for functions are proportional.
- Each request is executed by a dedicated container without preemption, and request queuing as well as deadline constraints are not explicitly modeled.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:21:50 UTC from IEEE Xplore. Restrictions apply.

C. Mathematical Model

There are K functions $\{F_1, F_2, \ldots, F_K\}$ in total. The cold start time of the function F_k is cs_k. The CPU and memory requirements of F_k are cpu_k and mem_k, respectively. The CPU and memory configurations cpu_k and mem_k are assumed to be proportional, which aligns with the configuration policies adopted by most cloud providers. During a duration TP, N_k requests $\{R_1^k, R_2^k, \ldots, R_{N_k}^k\}$ of the function F_k are received. Their arrival times are $\{a_1^k, a_2^k, \ldots, a_{N_k}^k\}$, and the execution times for these requests are $\{et_1^k, et_2^k, \ldots, et_{N_k}^k\}$. These requests are executed on a group of containers created for function F_k. One container executes one request at a time, and each request is executed with only one container. Requests of F_k can reuse these containers. In addition, all containers are deployed on PMs. All PMs are homogeneous, the CPU and memory configurations of which are cpu_{pm} and mem_{pm}, respectively. The decision variable $x_k \in [0, MRT]$ is the reuse time of F_k, where MRT is the maximum reuse time. Suppose that M_k containers $\{C_1^k, C_2^k, \ldots, C_{M_k}^k\}$ are created for F_k, the decision variable $y_{n,m}^k$ takes value 1 if the request R_n^k is assigned to container C_m^k. $y_{n,m}^k$ takes value 0, otherwise. Suppose that L PMs $\{PM_1, PM_2, \ldots, PM_L\}$ are created during TP in total, the decision variable $z_{m,l}^k$ takes the value 1 if the container C_m^k is deployed in PM PM_l. $z_{m,l}^k$ takes value 0, otherwise. In addition, an indicator function $I_{m,l}^k(t)$ indicates whether container C_m^k is deployed on PM_l at time t.

$$I_{m,l}^k(t) = \begin{cases} 1, & \text{if } ct_m^k < t < kt_m^k, \text{ and } z_{m,l}^k = 1, \\ 0, & \text{otherwise,} \end{cases} \tag{1}$$

The power consumption of machine PM_l at time t is denoted by $P_l(t)$, which is composed of a static power component P_{st} and a dynamic power component $P_l^{dy}(t)$, i.e., $P_l(t) = P_{st} + P_l^{dy}(t)$. The dynamic power is bounded by $0 \le P_l^{dy}(t) \le P_{\max} - P_{st}$, where $P_{\max}$ represents the maximum power consumption of PM_l. $P_l^{dy}(t)$ is determined by the resource utilization of PM_l. Specifically, the CPU utilization $u_l(t)$ at time t is defined as

$$u_l(t) = \frac{\sum_{k=1}^{K} \sum_{m=1}^{M_k} cpu_k \cdot I_{m,l}^k(t)}{cpu_{pm}}. \tag{2}$$

When all containers deployed on PM_l are in the running state, the dynamic power consumption can be approximated as a linear function of CPU utilization [30]. However, empirical studies show that the power consumption during the cold start phase is lower than that during normal execution, while the power consumption in the idle state is almost negligible [5]. Accordingly, the dynamic power of PM_l at time t can be expressed in a piecewise form as

$$P_l^{dy}(t) = \begin{cases} u_l(t) \cdot (P_{\max} - P_{st}), & \text{running state,} \\ \gamma\, u_l(t) \cdot (P_{\max} - P_{st}), & \text{cold start state,} \\ 0, & \text{idle state,} \end{cases} \tag{3}$$

where $\gamma \in (0, 1)$ is a discount factor reflecting the reduced dynamic power consumption during the cold start phase.

The objective of the studied problem is to minimize the total energy consumption TEC of the serverless computing platform, i.e.,

$$\begin{aligned} \min TEC &= \sum_{l=1}^{L} \int_{ct_l}^{kt_l} \left[P_{st} + P_l^{dy}(t) \right] \cdot dt \\ &= \sum_{l=1}^{L} P_{st}(kt_l - ct_l) + \sum_{k=1}^{K} (M_k \cdot cs_k \cdot \gamma \\ &\quad + \sum_{n=1}^{N_k} et_n^k) \cdot \frac{cpu_k}{cpu_{pm}} \cdot (P_{\max} - P_{st}), \end{aligned} \tag{4}$$

where ct_l is the creation time of PM_l, and kt_l is the killing time of PM_l. ct_l and kt_l can be calculated by

$$ct_l = \min_{k \in [1:K], m \in [1:M_k]} \{ct_m^k \cdot z_{m,l}^k\}, \tag{5}$$

$$kt_l = \min_{k \in [1:K], m \in [1:M_k]} \{kt_m^k \cdot z_{m,l}^k\}, \tag{6}$$

where ct_m^k is the creation time of container C_m^k, and kt_m^k is the killing time of C_m^k. ct_m^k and kt_m^k can be calculated by

$$ct_m^k = \min_{n \in [1:N_k]} \{a_n^k \cdot y_{n,m}^k\}, \tag{7}$$

$$kt_m^k = \max_{n \in [1:N_k]} \{(a_n^k + et_n^k) \cdot y_{n,m}^k + x_k + \rho_m^k \cdot cs_k\}, \tag{8}$$

where ρ_m^k is an indicator function, which means that container C_m^k just executes one request during TP, and it is calculated by

$$\rho_m^k = \begin{cases} 1, & \text{if } \sum_{n=1}^{N_k} y_{n,m}^k = 1, \\ 0, & \text{otherwise.} \end{cases} \tag{9}$$

When a request of F_k is received, a cold start occurs and a new container is launched if there is no idle container. The cold start probability CSP_k of F_k is calculated by

$$CSP_k = M_k / N_k, \tag{10}$$

which is an invocation-level definition consistent with prior work [31] and AWS Lambda.[5] Although alternative definitions exist (e.g., session-level or inter-arrival-based), they require unavailable information such as session identifiers or arrival-time distributions, and they tend to systematically overestimate or underestimate the cold-start probability. Given these limitations, we adopt M_k/N_k as a practical, trace-driven metric for cold-start probability. CSP_k is required to be below a specified threshold η_k, i.e.,

$$CSP_k \le \eta_k. \tag{11}$$

In addition, containers deployed on PM_l need to satisfy the CPU and memory capacity constraint at any time t, that is,

$$\sum_{k=1}^{K} \sum_{m=1}^{M_k} cpu_k \cdot I_{m,l}^k(t) \le cpu_{pm}, \forall l = 1, 2, \ldots, L, \tag{12}$$

$$\sum_{k=1}^{K} \sum_{m=1}^{M_k} mem_k \cdot I_{m,l}^k(t) \le mem_{pm}, \forall l = 1, 2, \ldots, L, \tag{13}$$

[5] https://aws.amazon.com/cn/blogs/compute/understanding-and-remediating-cold-starts-an-aws-lambda-perspective

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:21:50 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Bayesian Optimization based Energy Consumption Minimization (BOECM) Algorithm.

Input: Historical request traces $\{R_n^k\}$ for all functions F_k, $k \in [1, K]$
Output: Total energy consumption TEC

```
begin
    Initialize request queue Q by sorting all requests by arrival time;
    foreach k ∈ [1, K] do
        Define the subproblem;
        Determine the reuse time x_k using the improved Bayesian Optimization under the given RA and CD strategies;
    while Q ≠ ∅ do
        R ← Pop next request from Q;
        if An idle container of the corresponding function is available then
            Assign R to the idle container (RA);
        else
            Create new container C for R;
            Deploy container C to a suitable PM (CD);
    Calculate TEC based on the energy model (Equation 4);
    return TEC;
```

IV. Proposed Algorithm

In this section, the energy consumption model is deeply analyzed and the BOECM algorithm is introduced. It is difficult to determine the reuse times $\{x_k\}$ of SFs due to the continuous high-dimensional decision space. Therefore, this problem is decomposed into a group of subproblems based on the accurate energy consumption model in Section III. An improved Bayesian Optimization with dynamic search space is adopted to efficiently determine the optimal solution for each subproblem. Several heuristic strategies are proposed for request allocation and container deployment, respectively. The details of the BOECM algorithm are shown in Algorithm 1, which includes three modules, i.e., Reuse Time Determination (RTD), Request Allocation (RA) and Container Deployment (CD). It is worth noting that the reuse time determination process is conducted under a given set of request allocation and container deployment strategies, which define the execution process of the system. The reuse times obtained by RTD are optimized with respect to the system behavior induced by the selected RA and CD strategies, rather than being independent of them. All the details and pseudocodes of the three components are provided in the Appendix, available online.

A. Problem Decomposition

Since the reuse time determination problem consists of a large number of decision variables, it is decomposed into many subproblems consisting only one variable for simplification. Firstly, we prove that only one resource type (CPU or memory) may become the bottleneck of PMs. Secondly, we decompose the problem into subproblems considering single-dimensional bottleneck. Finally, we prove that the optimal solutions of the decomposed subproblems differ from the global optimum by a theoretically bounded gap in terms of energy minimization.

1) Single-Dimensional Bottleneck Analysis: We assume that all PMs are homogeneous, and each function F_k is deployed in containers with fixed, proportional resource configurations. Specifically, the ratio of CPU to memory allocated to each container remains constant across all functions, i.e., $cpu_k/mem_k = \lambda$ for all F_k. Although PMs are heterogeneous at a global scale, scheduling and resource management in cloud are typically performed within resource pools or instance families consisting of machines with the same hardware configurations. This abstraction is widely used in system-level analysis to simplify resource management [32]. Moreover, major commercial serverless platforms, such as AWS Lambda,[6] provision functions using predefined CPU-memory bundles, where the allocated CPU capacity scales proportionally with the configured memory size. Given the above proportionality, the resource usage on any PM at any time maintains the same ratio. Let $\mathcal{C}$ denote the set of containers deployed on a PM at time t, with total CPU and memory usage u_{cpu} and u_{mem}, respectively. The following equation is established:

$$u_{cpu} = \sum_{C \in \mathcal{C}} cpu_C = \lambda \cdot \sum_{C \in \mathcal{C}} mem_C = \lambda \cdot u_{mem} \tag{14}$$

Thus, the CPU and memory utilization of PMs always scale proportionally.

Since the total capacity of each homogeneous PM satisfies $cpu_{pm}/mem_{pm} = \lambda'$, that means only one resource type—either CPU or memory—can become saturated. CPU becomes the bottleneck if $\lambda > \lambda'$. Otherwise, memory becomes the bottleneck. It is not possible for both constraints to be active simultaneously. This property allows us to simplify the multidimensional PM resource constraint into a single-dimensional bottleneck representation without loss of generality.

The single-dimensional bottleneck property is derived under the assumptions of homogeneous PMs and proportional CPU-memory configurations across serverless functions. It serves as a necessary condition for enabling theoretical analysis in Section IV-A-3 instead of applying the proposed BOECM algorithm. In more general settings, such as heterogeneous PMs or mixed CPU-memory ratios, this property no longer strictly holds. In such cases, the theoretical bound does not apply, and the performance of BOECM can only be evaluated through experiments.

2) Subproblem Definition: In this section, the initial problem is decomposed into subproblems based on the CPU resource bottleneck. A similar analysis can be applied if memory becomes the bottleneck instead.

[6]https://docs.aws.amazon.com/lambda/latest/dg/configuration-memory.html

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:21:50 UTC from IEEE Xplore. Restrictions apply.

According to (12), for any physical machine PM_l,

$$\sum_{k=1}^{K}\sum_{m=1}^{M_k} cpu_k \cdot I_{m,l}^k(t) \le cpu_{pm}.$$

Divide both sides by cpu_{pm} and multiply both sides by P_{st},

$$\sum_{k=1}^{K}\sum_{m=1}^{M_k} P_{st}\frac{cpu_k}{cpu_{pm}} I_{m,l}^k(t) \le P_{st}.$$

Integrate both sides with respect to t over $[ct_l, kt_l]$,

$$\int_{ct_l}^{kt_l}\sum_{k=1}^{K}\sum_{m=1}^{M_k} P_{st}\frac{cpu_k}{cpu_{pm}} I_{m,l}^k(t)\,dt \le \int_{ct_l}^{kt_l} P_{st}\,dt.$$

Calculate the result of both sides,

$$\sum_{k=1}^{K}\sum_{m=1}^{M_k} P_{st}\frac{cpu_k}{cpu_{pm}}(kt_m^k - ct_m^k) z_{m,l}^k \le P_{st}\,(kt_l - ct_l).$$

Since one container is deployed on one PM, i.e., $\sum_{l=1}^{L} z_{m,l}^k = 1, \forall k = 1, 2, \ldots, K$ and $m = 1, 2, \ldots, M_k$. Therefore,

$$\sum_{k=1}^{K}\sum_{m=1}^{M_k} P_{st}\frac{cpu_k}{cpu_{pm}}(kt_m^k - ct_m^k) \le \sum_{l=1}^{L} P_{st}\,(kt_l - ct_l). \quad (15)$$

The right side of (15) is the static energy consumption of all PMs according to (4), which could be calculated approximately by the left side. In this case,

$$\begin{aligned} TEC \ge & \sum_{k=1}^{K}\sum_{m=1}^{M_k} P_{st}(kt_m^k - ct_m^k)\frac{cpu_k}{cpu_{pm}} \\ & + \sum_{k=1}^{K}(M_k \cdot cs_k \cdot \gamma + \sum_{n=1}^{N_k} et_n^k)\frac{cpu_k}{cpu_{pm}}(P_{\max} - P_{st}) \\ = & \sum_{k=1}^{K}\left[\sum_{m=1}^{M_k} P_{st}(kt_m^k - ct_m^k) + (M_k \cdot cs_k \cdot \gamma \right. \\ & \left. + \sum_{n=1}^{N_k} et_n^k)(P_{\max} - P_{st})\right]\frac{cpu_k}{cpu_{pm}}. \end{aligned} \quad (16)$$

Let

$$\begin{aligned} EC_k = & \left[\sum_{m=1}^{M_k} P_{st}(kt_m^k - ct_m^k) + (M_k \cdot cs_k \cdot \gamma \right. \\ & \left. + \sum_{n=1}^{N_k} et_n^k)(P_{\max} - P_{st})\right]\frac{cpu_k}{cpu_{pm}}. \end{aligned} \quad (17)$$

The value of EC_k is only related to the container creation and killing times of SF F_k instead of PMs. Therefore, EC_k can be regarded as the energy consumption caused by the function F_k. According to (7)–(8), the creation and killing times of containers is related to $\{x_k\}$ and $\{y_{n,m}^k\}$. Since the requests are allocated according to heuristic rules, that means they are indirectly determined by $\{x_k\}$. The kth subproblem is defined as

$$\begin{aligned} & \min EC_k \\ & \text{s.t. } (5)-(11) \end{aligned}$$

In each subproblem, the reuse time x_k is determined to minimize EC_k. The gap between TEC and $\sum_{k=1}^{K} EC_k$ is caused by the resource fragments when deploying containers.

3) Optimality Gap of Decomposed Subproblems: To analyze the theoretical gap between the global optimal solution and the optimal solutions of the decomposed, we consider the ratio between $\sum_{k=1}^{K} EC_k$ and TEC. The subproblem for each function F_k optimizes an estimated energy consumption EC_k, which includes both static and dynamic energy terms associated with its containers. In contrast, the total energy consumption TEC accounts for additional deployment factors, such as unused PM resources and idle times.

We focus on the static energy component, as it is the only part that differs between $\sum_{k=1}^{K} EC_k$ and TEC. Let the container-based estimate of static energy consumption be:

$$TEC_{static}^{\text{approx}} = \sum_{k=1}^{K}\sum_{m=1}^{M_k}(kt_m^k - ct_m^k)\cdot\frac{cpu_k}{cpu_{pm}}\cdot P_{st}$$

and let the actual static energy consumed by PMs be:

$$TEC_{static} = \sum_{l=1}^{L}(kt_l - ct_l)\cdot P_{st}$$

A factor $\alpha \in (0, 1]$ representing the ratio between these two values:

$$\alpha = \frac{TEC_{static}^{\text{approx}}}{TEC_{static}} = \frac{\sum_{k=1}^{K}\sum_{m=1}^{M_k}(kt_m^k - ct_m^k)\cdot\frac{cpu_k}{cpu_{pm}}}{\sum_{l=1}^{L}(kt_l - ct_l)}$$

A high value of α indicates high PM utilization with minimal idle resource fragments. The total estimated energy consumption is thus:

$$\sum_k EC_k = \alpha \cdot TEC_{static} + TEC_{dynamic}$$

Consequently, the optimality ratio of the subproblem-based solution satisfies:

$$\frac{\sum_k EC_k}{TEC} = \frac{\alpha \cdot TEC_{static} + TEC_{dynamic}}{TEC} > \alpha$$

Thus, we obtain a tight lower bound α of the subproblem decomposition.

The ratio α is defined as the ratio between the CPU-weighted container runtime and the actual active time of all physical machines (PMs). This can be equivalently expressed using continuous-time resource utilization:

$$\alpha = \frac{\sum_{l=1}^{L}\int_{ct_l}^{kt_l} u_l(t)\,dt}{\sum_{l=1}^{L}(kt_l - ct_l)} \quad (18)$$

This formulation shows that α represents the average CPU utilization across all PMs during their respective life-cycles. Suppose that each PM maintains a minimum CPU utilization throughout its active period, i.e., $u_l(t) \ge u_{\min} \in (0, 1]$ for all $t \in [ct_l, kt_l]$ and all l. Then, by applying the inequality to the integral, we obtain:

$$\int_{ct_l}^{kt_l} u_l(t)\,dt \ge u_{\min}\cdot(kt_l - ct_l)$$

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:21:50 UTC from IEEE Xplore. Restrictions apply.

Substituting this into (18), we derive:

$$\alpha \geq u_{\min}$$

This implies that as long as PMs are utilized above a certain threshold during their lifetime, the energy approximation based on container runtimes is guaranteed to stay within a bounded range of the actual energy consumption.

Fortunately, in high-load scenarios with efficient container deployment, the resource fragmentation on each PM is primarily determined by the largest container-to-PM CPU ratio. In the worst case, a PM may remain underutilized by up to:

$$\max_{k=1,\ldots,K} \left\{ \frac{cpu_k}{cpu_{pm}} \right\}$$

This leads to the following utilization lower bound:

$$u_l(t) \geq 1 - \max_k \left\{ \frac{cpu_k}{cpu_{pm}} \right\} \quad \Rightarrow \quad \alpha \geq 1 - \max_k \left\{ \frac{cpu_k}{cpu_{pm}} \right\}$$

As the capacity of each PM increases (i.e., $cpu_{pm} \to \infty$), the normalized container sizes $\frac{cpu_k}{cpu_{pm}}$ tend toward zero, reducing fragmentation. Therefore, $\alpha \to 1$ in the limit, implying that the subproblem-based solution asymptotically approaches the global optimum, providing a theoretical justification for the proposed problem decomposition method.

B. Reuse Time Determination (RTD)

Bayesian Optimization is a global optimization method used to find the optimal solution for complex, expensive, or challenging objective functions within a limited number of evaluations [33]. In this section, Bayesian Optimization with a dynamic space is adopted to efficiently determine the reuse time for each SF F_k by gradually narrowing the search scope.

To determine the optimal reuse time of F_k efficiently, the search space of Bayesian optimization is adjusted dynamically. The initial search space of x_k is set as $[0, MAT]$. Bayesian Optimization requires that x_k is feasible in the initial space. Therefore, the objective is defined as $g(x_k)$ instead of EC_k,

$$\min g(x_k) = EC_k \cdot (1 + \max\{0, \theta \cdot (CSP_k - \eta_k)\}), \quad (19)$$

where $\theta > 0$ is a punishment factor. When the cold start probability CSP_k is larger than the threshold η_k, the objective $g(x_k)$ increases quickly. This approach ensures that the optimal solution found satisfies the constraints. In addition, the left and right boundaries of the search space are updated with each iteration. Specifically, we modify the left and right boundaries as follows:

- *Left boundary:* Suppose that the set of observations is $\mathcal{D}_{1:t} = \{(x_k^i, CSP_k^i, EC_k^i)\}_{i=1}^t$, the maximal reuse time does not meet the cold start probability constraint is $x_k^\star = \arg\max_{x_k^i} \{x_k^i | CSP_k^i > \eta_k\}$. The left boundary LB_k after t observations is update by

$$LB_k = \max\{x_k^\star - EB, 0\}. \quad (20)$$

  where EB is the extension of boundaries.
- *Right boundary:* The best reuse time in $\mathcal{D}_{1:t}$ is $x_k^\star = \arg\min_{x_k^i} \{EC_k^i | CSP_k^i \leq \eta_k\}$. The right boundary RB_k is set a bit larger than $x_k^\star$, i.e.,

$$RB_k = \min\{x_k^\star + EB, MRT\}. \quad (21)$$

The left and right boundaries are updated according to the set of observations, and they are used for the next search. The above process repeats until the maximum iteration number MI is reached or there is a conflict between the boundaries, i.e., $RB_k \leq LB_k$.

C. Request Allocation (RA)

In this section, we propose several request allocation strategies to assign incoming requests to idle containers. Let $\mathcal{C}^k(t)$ denote the set of all idle containers for function F_k at time t. Each container $C_m^k \in \mathcal{C}^k(t)$ is characterized by its creation time ct_m^k and the finish time of its most recent request ft_m^k. We define the idle duration (or idle time) of a container at time t as $\delta_m^k = t - ft_m^k$.

Based on these characteristics, four container selection strategies are developed:

- *Earliest Creation Time First (ECTF):*

$$C_\star^k = \arg\min_{C_m^k \in \mathcal{C}^k(t)} \{ct_m^k\}$$

- *Latest Creation Time First (LCTF):*

$$C_\star^k = \arg\max_{C_m^k \in \mathcal{C}^k(t)} \{ct_m^k\}$$

- *Minimal Idle Duration First (MIIDF):*

$$C_\star^k = \arg\min_{C_m^k \in \mathcal{C}^k(t)} \{\delta_m^k\}$$

- *Maximal Idle Duration First (MAIDF):*

$$C_\star^k = \arg\max_{C_m^k \in \mathcal{C}^k(t)} \{\delta_m^k\}$$

In fact, the MIIDF and MAIDF strategies are conceptually related to the warmest-container-first and coldest-container-first scheduling policies studied in prior work [17]. In this paper, we adopt these strategies as representative request allocation heuristics to analyze their impact within a container-state-aware energy optimization framework. Given the reuse time, LCTF and MIIDF tend to reduce the number of active containers, whereas ECTF and MAIDF favor reducing the probability of cold starts.

D. Container Deployment (CD)

When a new container is created, it must be deployed on a PM. Let $\mathcal{Q}(t)$ denote the set of currently active PMs at time t that satisfy the resource capacity constraints in (12) and (13). If $\mathcal{Q}(t) = \emptyset$, a new PM is created for deployment. Otherwise, the container is assigned to one of the existing PMs in $\mathcal{Q}(t)$.

Since multiple candidate PMs may satisfy the constraints, we propose the following container deployment strategies:

- *First Fit (FF):* Sort PMs by creation time in ascending order. Select the first PM in the list.
- *Minimum Remaining Resources First (MRRF):* For each $PM_l \in \mathcal{Q}(t)$, let PM_l^{cpu} and PM_l^{mem} denote the percentage of CPU and memory remaining after placing the

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:21:50 UTC from IEEE Xplore. Restrictions apply.

container. Choose:

$$PM_\star = \underset{PM_l \in \mathcal{Q}(t)}{\arg\min} \{PM_l^{cpu} + PM_l^{mem}\}$$

- *Resource Balancing First (RBF):* Balance CPU and memory utilization by selecting:

$$PM_\star = \underset{PM_l \in \mathcal{Q}(t)}{\arg\min} \left\{ \left| \frac{PM_l^{cpu} + \epsilon}{PM_l^{mem} + \epsilon} - 1 \right| \right\}$$

where ϵ is a small positive constant for numerical stability.

- *Comprehensive Evaluation (CE):* A hybrid of MRRF and RBF. Select:

$$PM_\star = \underset{PM_l \in \mathcal{Q}(t)}{\arg\min} \left\{ (PM_l^{cpu} + PM_l^{mem}) \cdot \left| \frac{PM_l^{cpu} + \epsilon}{PM_l^{mem} + \epsilon} - 1 \right| \right\}$$

Although all CD strategies aim to reduce the number of activated PMs and the associated static energy consumption, they differ in how placement decisions are made to achieve this objective. FF attempts to minimize the activation of new PMs by placing containers on earlier-activated PMs, whereas MRRF achieves resource consolidation by packing containers onto PMs with minimal remaining capacity. In contrast, RBF emphasizes resource balance to mitigate fragmentation and preserve future placement flexibility, while CE explicitly trades off consolidation and resource feasibility by jointly considering both factors.

E. Computational Time Complexity Analysis

The computational time complexity of the RA and CD procedures is linear in the total number of historical requests. Assuming constant-time lookup for idle containers and PMs (e.g., via efficient data structures), both RA and CD have a complexity of: $O(\sum_{k=1}^{K} N_k)$ For Reuse Time Determination (RTD), Bayesian Optimization is performed independently for each serverless function F_k. Each BO iteration involves simulating the execution of F_k's request history, which costs $O(N_k)$. Given that the maximum number of BO iterations is MI, the overall complexity of RTD is: $O(\sum_{k=1}^{K} MI \cdot N_k) = O(MI \cdot \sum_{k=1}^{K} N_k)$

F. Applicability and Extensions

The theoretical analysis presented in this paper relies on the single-dimensional bottleneck assumption and therefore does not extend to heterogeneous PMs or mixed CPU-memory ratios. Nevertheless, the proposed BOECM algorithm can still and executed as a heuristic method with some modifications.

First, heterogeneous PMs may have diverse static power P_{st}, peak power $P_{\max}$, and resource capacity cpu_{pm}, the weighted average values of these parameters can be used for problem decomposition. This approximation allows BOECM to capture the features of a heterogeneous cluster. Second, during container deployment, blindly creating a new PM when existing PMs cannot accommodate additional containers may lead to suboptimal resource utilization in heterogeneous environments. Instead, the selection of a new PM type should be guided by the aggregated CPU and memory demands of currently active containers, so as to reduce resource fragmentation.

V. Experimental Results

To the best of our knowledge, the container-state-aware energy minimization problem with cold start probability constraints has not been addressed in the existing literature. To identify the most effective solution for this problem, all algorithmic components of the proposed BOECM algorithm are calibrated on a large set of randomly generated instances. We compare the calibrated BOECM algorithm with several existing methods designed for similar problems based on the real-world Azure workload trace. In both the component calibration and algorithm comparison phases, function reuse times are determined using a training set, and the performance is evaluated on a separate test set containing the same functions. All algorithms are implemented in Python and executed on a machine with an Intel Core i7-7500 U CPU @ 2.70 GHz and 12 GB of RAM. In the following, the detailed experimental setup and experimental results are introduced.

A. Experimental Setup

The experimental setup includes evolution metrics, baselines, resource configuration, and dataset.

1) Evolution Metrics: Three metrics are adopted for algorithm evaluation and comparison:

- RPD (Relative Percentage Deviation), which is defined by

$$RPD(\%) = \frac{TEC(A) - TEC^\star}{TEC^\star} \times 100\%, \tag{22}$$

$TEC(A)$ is the total energy consumption obtained by algorithm A, while $TEC^\star$ is the minimal total energy consumption obtained among all compared algorithms on the same test instance.

- SVD (Service Violation Degree), which is defined by

$$SVD = CSP(A) - \eta, \tag{23}$$

where $CSP(A)$ is the cold start probability obtained by algorithm A, while η is the pre-determined threshold.

- CS (Convergence Speed). The convergence speed of the proposed BOECM algorithm is compared to baselines.

The ANOVA (multi-factor analysis of variance) technique is adopted to analyze the RPD and SVD metrics. The three main hypotheses(normality, homoscedasticity, and independence of the residuals) are checked from the residuals of the experiments. All three hypotheses are acceptable from the analysis.

2) Resource Configuration: The containers and PMs are configured in terms of Ali cloud.[7] The CPU configuration of each SF ranges from 0.05 to 16 cores and must be a multiple of 0.05. The CPU and memory configurations of PMs are set according to the ECS bare metal server (the specific model is "ebmc6me".) provided by Ali cloud, i.e., $cpu_{pm} = 64, mem_{pm} = 192$ GB.

[7]https://help.aliyun.com/zh/fc/user-guide/instance-types-and-instance-modes

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:21:50 UTC from IEEE Xplore. Restrictions apply.

TABLE I
STATIC AND MAXIMUM POWER OF DIFFERENT PMS

CPU Model	Cores	Memory(GB)	P_{st}(Wh)	P_{max}(Wh)
E5-2630	12	128	99.6	325
E5430	8	16	166	265
E5507	8	8	67	218
E5-2620	12	32	70	300
E5645	12	16	63.1	200
E5-2650	16	24	52.9	215
E5-2651	12	32	57.5	178
E5-2670	16	24	54.1	243
E5540	4	72	151	312
X5560	8	128	133	288
E5-2665	8	256	117	314
X5650	12	64	80.1	258
Average Power			92.61	259.67

Different types of PMs have different static and maximum power. Diverse static and maximum power values of Amazon cloud infrastructure [34] are shown in Table I. For simplicity, we use the average values of static power and maximum power in this paper, i.e., $P_{st} = 92.61$Wh, $P_{\max} = 259.67$Wh. The discounted factor of cold start is set as $\gamma = 0.7$ according to [5].

3) Baselines: To the best of our knowledge, there is no existing algorithm for the considered problem under study. To test the performance of the proposed BOECM algorithm, several algorithms for similar problems are adopted for comparison. The baselines include heuristic algorithms, such as Fixed Reuse Time (FRT) adopted in AWS Lambda, HHP [9], and RAEF [5]. In addition, BOECM is also compared with several global optimization algorithms, such as Genetic Algorithm (GA), Differential Evolution (DE), Evolution Strategy (ES), and classical Bayesian Optimization (BO). We further include the prediction-based PS (Perspective Scheduling) strategy proposed in [14]. To ensure a fair comparison under a fixed reuse-time setting, the average predicted reuse time learned from the training set is applied to the test set. The above algorithms are compared using RPD and SVD metrics. All global optimization algorithms are compared over 100 iterations. The global optimization algorithms are implemented by the toolkit Geatpy.[8] Bayesian Optimization is implemented using the `SKOPT` toolkit,[9] with all hyper-parameters set to default values. The BOECM algorithm uses fixed parameters, where the penalty factor is set to $\theta = 10$ and the extended boundary is $EB = 10$.

4) Dataset: In this paper, to calibrate the parameters and validate the theoretical lower bound of the proposed BOECM algorithm, synthetic workloads are randomly generated. The simulation duration is set to $TP = 2$ days (2 days for training, and 2 days for testing), and the number of serverless functions is fixed at $K = 20$. The arrival process of each SF follows a non-homogeneous Poisson process. Arrival rates λ_k are randomly drawn from the interval $[0.01, 1.0]$, and vary over time according to sine wave functions whose periods are uniformly distributed in $[1, 4]$ hours. Each SF has a cold start time sampled from a uniform distribution $\mathcal{U}(0.1, 60)$ seconds. The execution time

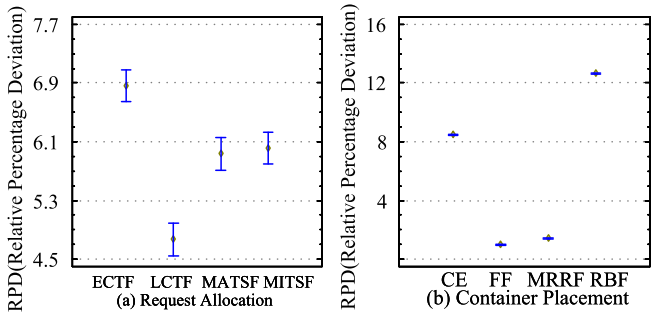


Fig. 2. Mean plots of the different components of the BOECM algorithm with 95.0% Tukey HSD confidence level intervals.

of each request for function F_k follows a normal distribution $e_n^k \sim \mathcal{N}(\mu_k, \sigma_k^2)$, where $\mu_k \sim \mathcal{U}(0.1, 60)$ and $\sigma_k \sim \mathcal{U}(0.1, 1)$, all in seconds. The maximum reuse time is set to one hour. This data generation process is repeated 50 times. For each training and testing dataset, the cold start probability threshold η_k is selected from the set $\{0.05, 0.1, 0.15, 0.2, 0.25\}$, resulting in a total of 250 test instances.

To compare the performance of the proposed BOECM algorithm with the baseline methods, experiments are conducted based on the Azure Function Invocation Trace 2021[10] dataset. First, we preprocess the raw data by filtering out applications with an arrival rate lower than 0.001 and those with unbalanced request distributions (where the proportion of requests in the first half of the global time window falls outside the range of [40% , 60%]). After filtering, a total of 14 applications (each can be regarded as a function) are retained. This unified test workload is used to quantitatively evaluate the performance of all compared algorithms.

B. Component Calibration

Two algorithmic components are subject to calibration: request allocation (RA) and container deployment (CD). For RA, four strategies are considered: ECTF, LCTF, MIIDF, and MAIDF. For CD, four strategies are evaluated: FF, MRRF, RBF, and CE. This results in a total of $4 \times 4 = 16$ possible strategy combinations. Given 250 benchmark instances, a total of 4000 simulation tests are conducted.

The mean RPD values along with 95% Tukey HSD confidence intervals are illustrated in Fig. 2. As shown in Fig. 2(a), LCTF consistently achieves the lowest RPD and statistically outperforms the other RA strategies. Fig. 2(b) indicates that FF and MRRF exhibit similar performance, with FF showing a slight advantage. LCTF tends to concentrate requests on a smaller subset of recently created containers, allowing older idle containers to expire earlier under the same reuse-time configuration compared to other strategies. As a result, it reduces the number of simultaneously kept-alive containers and lowers the static energy consumption incurred by PMs for maintaining idle capacity. FF further reduces the number of active PMs by preferentially placing new containers on earlier-launched machines, thereby shortening the aggregate PM active-time.

[8]http://geatpy.com/index.php/home/
[9]https://scikit-optimize.github.io/stable/
[10]https://github.com/Azure/AzurePublicDataset

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:21:50 UTC from IEEE Xplore. Restrictions apply.

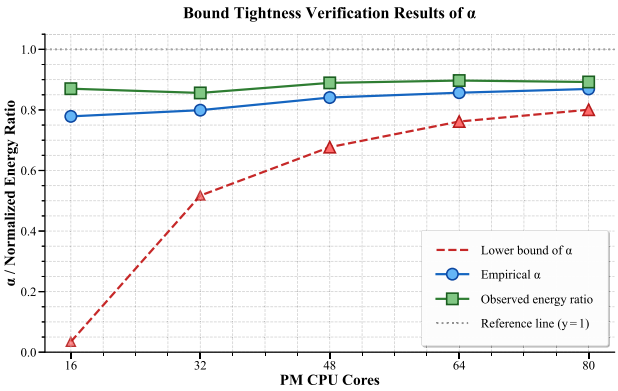


Fig. 3. Bound tightness verification.

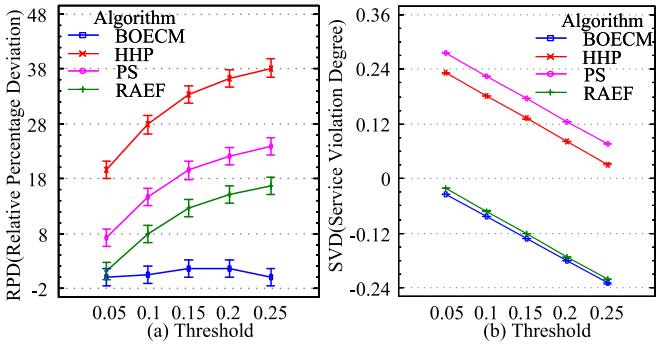


Fig. 4. Interaction plots of the compared heuristic algorithms and thresholds with 95.0% Tukey HSD confidence level intervals.

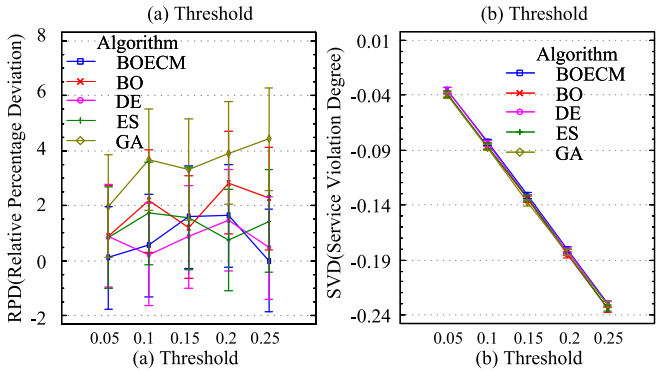


Fig. 5. Interaction plots of the compared global optimization algorithms and thresholds with 95.0% Tukey HSD confidence level intervals.

Both strategies significantly outperform RBF and CE. Based on these results, LCTF and FF are selected as the default RA and CD strategies for the BOECM algorithm in subsequent experiments.

C. Validation of the Theoretical Bound

To empirically validate the tightness and practical usefulness of the proposed α bound under the single-dimensional bottleneck setting, we evaluate the bound by varying the PM CPU capacity and reporting three quantities: the theoretical lower bound of α, the measured α value (defined as the ratio between the approximated and the actual static energy consumption), and the ratio between the approximated total energy ($\sum_{k=1}^{K} EC_k$) and the actual total energy. The CPU cores of PM range in $\{16, 32, 48, 64, 80\}$, the constraint of cold start probability is set as 0.1. As shown in Fig. 3, the measured α values are consistently above the theoretical lower bound and closely follow the ratio given by the approximated-to-actual total energy. In all tested configurations, the sum of subproblem objectives provides a tight approximation of the global objective, reaching approximately 93.6% of the total energy, while the measured α values remain around 83.2% . These results demonstrate that the proposed bound is tight in practice.

D. Comparisons With Existing Methods

The interactions between cold start probability thresholds and the compared heuristic algorithms, with 95% Tukey HSD confidence intervals, are shown in Fig. 4. As shown in Fig. 4(a), BOECM consistently achieves the lowest RPD across all thresholds, significantly outperforming HHP, PS, and RAEF in terms of energy consumption. In contrast, the RPDs of HHP and PS increase rapidly as the threshold becomes more relaxed. Although RAEF is designed with energy awareness, it exhibits moderate but inferior performance. In particular, due to its lack of explicit cold-start constraint awareness, RAEF shows performance comparable to BOECM under stringent thresholds, but fails to adapt its reuse time accordingly as the threshold becomes more relaxed. Fig. 4(b) shows that BOECM and RAEF maintain negative SVD values under all thresholds, strictly satisfying the cold-start probability constraints, whereas HHP and PS violate

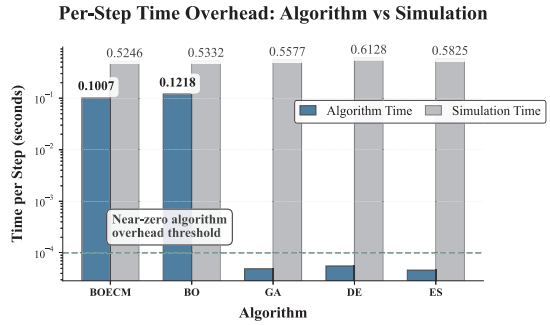


Fig. 6. Per-step time overhead.

the constraints under stringent thresholds. Overall, RAEF is conservative with limited energy savings, and BOECM effectively balances energy efficiency and constraint satisfaction across all evaluated settings.

Figs. 5, 6, and 7 compare the proposed BOECM algorithm with several global optimization algorithms. As shown in Fig. 5(b), all algorithms successfully meet the cold-start probability constraints, as their SVD values remain below zero across all threshold settings. Fig. 5(a) shows that BOECM achieves energy consumption performance comparable to BO, DE, GA, and ES, with no statistically significant degradation in RPD. However, Figs. 6 and 7 reveal a substantial advantage in convergence efficiency. Specifically, BOECM incurs higher

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:21:50 UTC from IEEE Xplore. Restrictions apply.

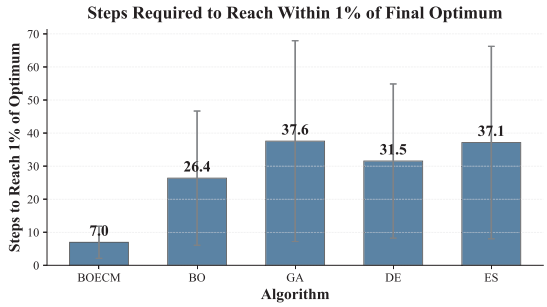


Fig. 7. Convergence steps histogram.

per-step algorithm overhead than other algorithms except BO. However, BOECM converges significantly faster, requiring far fewer steps to reach within 1% of the final optimum compared to BO, GA, DE, and ES. Moreover, during reuse-time optimization, the dominant cost is the simulation overhead rather than the algorithmic overhead. As a result, BOECM achieves shorter overall optimization time in practice. These results indicate that BOECM attains competitive energy optimization quality while substantially reducing optimization time, making it more suitable for practical reuse-time optimization under realistic system constraints.

VI. Conclusion

This paper studies an container-state-aware energy minimization problem for serverless functions with cold start probability constraints. The problem is decomposed into a group of subproblems. The proposed BOECM algorithm adopts an improved Bayesian optimization with a dynamic search space to efficiently search for the reuse times. Several heuristics are proposed for request allocation and container deployment. Experimental results show that the BOECM outperforms the compared algorithms in minimizing energy consumption, meeting the cold start probability constraints, and the convergence speed.

There are still many topics worth studying in the future. For example, this paper sets a fixed reuse time for each SF. However, the invocation pattern of SFs may fluctuate over time. Therefore, a real-time reuse time adjustment method may be more effective. Nevertheless, directly applying BOECM in such a dynamic setting may incur non-negligible overhead. How to design lightweight dynamic adjustment methods for reuse time remains an open problem and deserves further study. In addition, integrating the proposed BOECM algorithm with predictive autoscaling techniques, as well as request queuing and deadline-aware scheduling mechanisms, and incorporating real-world measurements to build more accurate energy consumption models, is a promising direction for further reducing cold-start latency and energy consumption.

References

[1] E. Jonas et al., "Cloud programming simplified: A Berkeley view on serverless computing," 2019, *arXiv:1902.03383*.

[2] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, "Peeking behind the curtains of serverless platforms," in *Proc. USENIX Annu. Tech. Conf.*, 2018, pp. 133–146.

[3] M. Zhang, F. Wang, Y. Zhu, J. Liu, and Z. Wang, "Towards cloud-edge collaborative online video analytics with fine-grained serverless pipelines," in *Proc. 12th ACM Multimedia Syst. Conf.*, 2021, pp. 80–93.

[4] C. Denninnart and M. A. Salehi, "Harnessing the potential of function-reuse in multimedia cloud systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 3, pp. 617–629, Mar. 2022.

[5] X. Jia and L. Zhao, "RAEF: Energy-efficient resource allocation through energy fungibility in serverless," in *Proc. IEEE 27th Int. Conf. Parallel Distrib. Syst.*, 2021, pp. 434–441.

[6] N. Mahmoudi and H. Khazaei, "Performance modeling of serverless computing platforms," *IEEE Trans. Cloud Comput.*, vol. 10, no. 4, pp. 2834–2847, Fourth Quarter 2022.

[7] M. Szalay, P. Matray, and L. Toka, "Real-time FAAS: Towards a latency bounded serverless cloud," *IEEE Trans. Cloud Comput.*, vol. 11 no. 2 pp. 1636–1650 Second Quarter 2023.

[8] A. Wang et al., "FaaSNet: Scalable and fast provisioning of custom serverless container runtimes at alibaba cloud function compute," in *Proc. 2021 USENIX Annu. Tech. Conf.*, 2021, pp. 443–457.

[9] M. Shahrad et al., "Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider," in *Proc. USENIX Annu. Tech. Conf.*, 2020, pp. 205–218.

[10] P. Vahidinia, B. Farahani, and F. S. Aliee, "Mitigating cold start problem in serverless computing: A reinforcement learning approach," *IEEE Internet Things J.*, vol. 10, no. 5, pp. 3917–3927, Mar. 2023.

[11] A. U. Gias and G. Casale, "COCOA: Cold start aware capacity planning for function-as-a-service platforms," in *Proc. 28th Int. Symp. Modeling, Analysis, Simul. Comput. Telecommun. Syst.*, 2020, pp. 1–8.

[12] A. Suresh, G. Somashekar, A. Varadarajan, V. R. Kakarla, H. Upadhyay, and A. Gandhi, "Ensure: Efficient scheduling and autonomous resource management in serverless environments," in *Proc. 2020 IEEE Int. Conf. Autonomic Comput. Self-Organizing Syst.*, 2020, pp. 1–10.

[13] Z. Zhang, T. Wang, A. Li, and W. Zhang, "Adaptive auto-scaling of delay-sensitive serverless services with reinforcement learning," in *Proc. IEEE 46th Annu. Comput., Softw., Appl. Conf.*, 2022, pp. 866–871.

[14] Q. Hu, H. Li, and E. Nikougoftar, "Mitigating cold start problem in serverless computing using predictive pre-warming with machine learning," *Computing*, vol. 107, no. 1, 2025, Art. no. 27.

[15] X. Cai et al., "Incendio: Priority-based scheduling for alleviating cold start in serverless computing," *IEEE Trans. Comput.*, vol. 73, no. 7, pp. 1780–1794, Jul. 2024.

[16] M. Golec et al., "Master: Machine learning-based cold start latency prediction framework in serverless edge computing environments for industry 4.0," *IEEE J. Sel. Areas Sensors*, vol. 1, pp. 36–48, 2024.

[17] A. Choupani, S. Azizi, and M. S. Aslanpour, "Joint resource autoscaling and request scheduling for serverless edge computing," *Cluster Comput.*, vol. 28, no. 3, 2025, Art. no. 171.

[18] Z. Xu, H. Zhang, X. Geng, Q. Wu, and H. Ma, "Adaptive function launching acceleration in serverless computing platforms," in *Proc. IEEE 25th Int. Conf. Parallel Distrib. Syst.*, 2019, pp. 9–16.

[19] X. Liu et al., "FaaSLight: General application-level cold-start latency optimization for function-as-a-service in serverless computing," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, pp. 1–29, 2023.

[20] I. E. Akkus et al., "SAND: Towards high-performance serverless computing," in *Proc. Usenix Annu. Tech. Conf.*, 2018, pp. 923–935.

[21] N. Daw, U. Bellur, and P. Kulkarni, "Xanadu: Mitigating cascading cold starts in serverless function chain deployments," in *Proc. 21st Int. Middleware Conf.*, 2020, pp. 356–370.

[22] J. Shen, T. Yang, Y. Su, Y. Zhou, and M. R. Lyu, "Defuse: A dependency-guided function scheduler to mitigate cold starts on faas platforms," in *Proc. IEEE 41st Int. Conf. Distrib. Comput. Syst.*, 2021, pp. 194–204.

[23] V. M. Bhasi, J. R. Gunasekaran, P. Thinakaran, C. S. Mishra, M. T. Kandemir, and C. Das, "Kraken: Adaptive container provisioning for deploying dynamic dags in serverless platforms," in *Proc. ACM Symp. Cloud Comput.*, 2021, pp. 153–167.

[24] C. Wu, R. Buyya, and K. Ramamohanarao, "Cloud pricing models: Taxonomy, survey, and interdisciplinary challenges," *ACM Comput. Surv.*, vol. 52, no. 6, pp. 1–36, 2019.

[25] H. Yang, Q. Chen, M. Riaz, Z. Luan, L. Tang, and J. Mars, "PowerChief: Intelligent power allocation for multi-stage applications to improve responsiveness on power constrained CMP," in *Proc. 44th Annu. Int. Symp. Comput. Archit.*, 2017, pp. 133–146.

[26] J. R. Gunasekaran, P. Thinakaran, N. C. Nachiappan, M. T. Kandemir, and C. R. Das, "Fifer: Tackling resource underutilization in the serverless ERA," in *Proc. 21st Int. Middleware Conf.*, 2020, pp. 280–295.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:21:50 UTC from IEEE Xplore. Restrictions apply.

[27] S. H. Rastegar, H. Shafiei, and A. Khonsari, "EneX: An energy-aware execution scheduler for serverless computing," *IEEE Trans. Ind. Informat.*, vol. 20, no. 2, pp. 2342–2353, Feb. 2024.
[28] P. Zuk and K. Rzadca, "Reducing response latency of composite functions-as-a-service through scheduling," *J. Parallel Distrib. Comput.*, vol. 167, pp. 18–30, 2022.
[29] A. Mampage, S. Karunasekera, and R. Buyya, "Deep reinforcement learning for application scheduling in resource-constrained, multi-tenant serverless computing environments," *Future Gener. Comput. Syst.*, vol. 143, pp. 277–292, 2023.
[30] A. A. Khan, M. Zakarya, R. Buyya, R. Khan, M. Khan, and O. Rana, "An energy and performance aware consolidation technique for containerized datacenters," *IEEE Trans. Cloud Comput.*, vol. 9, no. 4, pp. 1305–1322, Fourth Quarter 2021.
[31] X. Chai et al., "Fork in the road: Reflections and optimizations for cold start latency in production serverless systems," in *Proc. 19th USENIX Symp. Operating Syst. Des. Implementation*, 2025, pp. 199–218.
[32] X. Ding et al., "Towards VM rescheduling optimization through deep reinforcement learning," in *Proc. 20th Eur. Conf. Comput. Syst.*, 2025, pp. 686–701.
[33] J. Snoek et al., "Scalable Bayesian optimization using deep neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 2171–2180.
[34] A. A. Khan, M. Zakarya, I. U. Rahman, R. Khan, and R. Buyya, "HeporCloud: An energy and performance efficient resource orchestrator for hybrid heterogeneous cloud computing environments," *J. Netw. Comput. Appl.*, vol. 173, 2021, Art. no. 102869.
[35] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. De Freitas, "Taking the human out of the loop: A review of Bayesian optimization," *Proc. IEEE*, vol. 104, no. 1, pp. 148–175, Jan. 2016.



Jinquan Zhang is currently a lecturer with the School of Computer Science, Guangdong University of Technology, China. His research interests include cloud computing resource management, serverless computing, and scheduling optimization. He has published more than ten research papers in well-known international journals and conferences, including *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Transactions on Services Computing*, and *Future Generation Computer Systems*.



Han Cao received the BScdegree from the School of Cyber Science and Engineering, Southeast University, Nanjing, China, in 2022. He is currently working toward the master's degree with the School of Computer Science and Engineering, Southeast University, Nanjing, China. His research interests focus on edge computing, serverless optimization, and cloud resource scheduling.



Long Chen received the BSc and PhD degrees in computer science and engineering from Southeast University, Nanjing, China, in 2009 and 2018 respectively. He is currently a lecturer in the Department of Computer Science and Engineering with Southeast University, Nanjing, China. He has published more than 20 papers in international journals and conferences, such as *IEEE Transactions on Services Computing*, *IEEE Transactions on Cloud Computing*, *IEEE Transactions on Automation Science and Engineering,*. His main interest includes task scheduling in cloud computing, service-oriented computing, evolutionary computation, data normalization in smart building.



Siyuan Li admitted to southeast University, Nanjing, China, in 2021. He is currently working toward the undergraduate majoring in software engineering of Southeast University, Nanjing, China. His research interests focus on Big Data, cloud computing and cloud resource scheduling.



Xiaoping Li (Senior Member, IEEE) received the BSc and MSc degrees in applied computer science from the Harbin University of Science and Technology, in 1993 and 1999, respectively, and the PhD degree in applied computer science from the Harbin Institute of Technology, in 2002. He is a distinguished professor with the School of Computer Science and Engineering, Southeast University, Nanjing, China. He is the author or co-author over more than 100 academic papers, some of which have been published in international journals such as *IEEE Transactions on Computers*, *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Transactions on Services Computing*, *IEEE Transactions on Cybernetics*, *IEEE Transactions on Automation Science and Engineering*, *IEEE Transactions on Cloud Computing*, *IEEE Transactions on Systems, Man and Cybernetics: Systems*, *Information Sciences*, *Omega*, *European Journal of Operational Research*, *International Journal of Production Research*, *Expert Systems with Applications*, and *Journal of Network and Computer Applications*. His research interests include Scheduling in Cloud Computing, Scheduling in Cloud Manufacturing, Service Computing, Big Data and Machine Learning.



Bing Ai received the PhD degree from Southeast University, Nanjing, China, in 2022. He is currently an assistant professor with Hefei University. He has authored and co-authored more than 10 papers in refereed journals and conference proceedings. His current research interests include multi-agent reinforcement learning, federated learning, UAV-assisted resource allocation, and game theory.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:21:50 UTC from IEEE Xplore. Restrictions apply.